CHAPTER 1
INTRODUCTION

This project is focused on implementation of 3D lung digital twin, which is intended to enable interaction with lungs and observation of their behavior within the virtual reality environment. It is obvious that respiratory mechanism (breathing process, for instance) plays an extremely significant role for medical education; however, traditional technologies including CT do not provide dynamic pictures of organs' functioning. As a result, it is rather problematic for future doctors to understand how to perform certain actions concerning the dynamic process of lungs' function in real medical conditions. With the help of the digital twin, it is possible to create a picture of a dynamic model, which is similar to real life.

In order to solve the described problem, it is suggested to combine the techniques of medical imaging and CT scanning of the lungs, as well as simulate dynamic physiological processes with the use of BioGears engine. Real time visualization is planned to be performed with the help of Unity program. Thus, lungs' volume, SpO2 and respiratory rate can be simulated through the 3D model.

For the purpose of implementing the above-mentioned system, it is necessary to reach the point of creating a model that will combine medical imaging with real-time physiological simulations and visualization of lungs' breathing process.

CHAPTER 2
PROBLEM DEFINITION

The standard procedures for generating the image of the lungs using the computer tomography scanning techniques have some limitations because they create static images of the lungs. In this respect, it becomes quite difficult to understand dynamic processes which occur when someone inhales or exhales. Consequently, it becomes rather difficult to figure out how things happen in the lungs depending on different variables related to breathing processes.

Furthermore, there is no interactive tool used in order to facilitate the learning process. Medical physiological simulations and visualization tools usually operate independently and hence correlation of real-time physiological data and graphical data becomes rather problematic.

In view of all the above, there is need to develop an integrated system for medical imaging, simulation, and visualization. The biggest advantage that comes with this system is its ability to enhance visualization and analysis of the respiratory system.

CHAPTER 3
DATA

3.1 Overview

The type of data that will be used in this study will mainly be obtained from image databases. Image databases include images of lungs taken through CT scans, therefore data concerning their structure can be obtained from them. The physiologic data includes respiratory rate, oxygen saturation levels, and lung volume. Hence, by using the data on both structure and physiology, the system will be able to give information on both structure and physiology of lungs.

3.2 Dataset

In the case of this dataset, it consists of CT scan images which provide cross-sectional images of the lungs. Such kind of images is always associated with medical data which is stored in files known as DICOM. The images from the CT scans are analyzed to extract the image of the lungs portion in three-dimensional form. Besides, simulation data is generated by using the BioGears software for performing the physiological process simulation, like breathing.

3.3 Data Preprocessing

The data collected from the system undergoes different forms of preprocessing before applying it anywhere. Initially, the CT-scan images are cleaned, and the lungs are segmented from the other components within the image. The resulting data is then converted into the form of 3D models that can be observed based on the user's needs. Furthermore, the 3D models undergo modifications to ensure that they perform optimally within the Unity interface. In addition, the physiological parameters are adjusted to correspond with the 3D model to create a lifelike breathing motion.

CHAPTER 4
DESIGN DETAILS

4.1 Novelty

The suggested system will be a novel system that will merge 3D lung visualization and physiological simulation in the virtual reality environment. Conventional methods use fixed CT scans, which are not dynamic in following the breathing motion. This system fills that gap with real time, parameter driven lung behavior.

Some important features of novelty are:

- Simulation of dynamic breathing as opposed to static imaging.
- Combination of physiological data (RR, SpO 2, lung volume)
- Live communication in a VR.
- Internal lung behavior to user action visual mapping.

How this is different to current approaches:

Traditional systems are disconnected between imaging and simulation. In this case, they are combined and the user can see the direct relationship between variation of physiological conditions and the motion of lungs.

Example:

In the simulation, the lung model shows an intuitive explanation of breathing patterns as respiratory rate increases, which is reflected by faster expansion and contraction of the lungs.

4.2 Innovativeness:

The system is pioneering in its manner of integrating a variety of technologies- medical imaging, simulation engines and real-time visualization into one interactive platform. This makes it a more interactive and productive tool of learning and analyzing as opposed to traditional systems.

Innovative elements include:

- BioGears being used to simulate physiological data in real-time.
- 3D rendering which is unity-based to visualize smoothly.
- Simulation parameters are interactively controlled.
- Export of simulation output into animation code.

The reason why this is innovative is as follows:

Majority of the systems either simulate data or visualize. This system not only does this in a real time, but also links them to each other.

Impact of innovation:

This method enhances respiratory behavior knowledge and create opportunities to interactive medical education and training.

4.3 Interoperability

The system has been built keeping interoperability in consideration therefore the various components can be used freely. It promotes integration of simulation engines, 3D modeling tools, and visualization tools.

Interoperability features:

- Standard formats (e.g., FBX 3D models) used.
- Scripting of BioGears outputs into Unity.
- Scalable architecture to add new components in the future.
- It has the ability to be compatible with VR systems and conventional computing hardware.

System flexibility:
With each module (simulation, visualization, UI) being able to be changed or substituted without impacting the rest of the system.

Example:
The visualization module can be retained with the simulation engine current or upgraded, thus extending its usability in the long-term.

4.4 Performance

The system is built in such a way that it provides a smooth real time performance which is required in an interactive VR based application. Efficient rendering and scripting techniques are used to maintain responsiveness.

Performance aspects considered include:

- Real-time animation without latency.

- Optimized polygonal 3D model.
- C# efficient scripting for updating of parameter values.
- Fixed simulation frame rate.
- Methods applied in optimization include:
- Decimating mesh in order to reduce computational effort.
- Controlled update cycles during animations.
- Lightweight simulation output data processing.

Conclusion: The system guarantees efficiency even with updates to physiological parameter values.

4.5 Security

Security is an important consideration, especially when dealing with medical-related data. Although the present system handles simulated data, it is set to accommodate secure data handling in the future.

Security aspects include:

- Limited access to controls of the system.
- Secure management of simulation data.
- Future patient data integration design considerations.
- Possibility of transmission of encrypted data in longer versions.

Future scope:

In case they are combined with actual hospital systems, further security protocols like authentication and secure storage may be adopted.

Importance:

To be adopted in real medical applications, it is important to ensure data privacy and integrity.

4.6 Reliability

The system is designed to ensure reliable performance during simulation and visualization. The BioGears physiology engine is incorporated to allow the constant and scientifically-based calculation of respiratory parameters, including oxygen saturation, respiratory rate, and lung volume. Unity offers a fixed rendering backdrop, so the expansion and contraction of the lungs can be easily visualized.

Reliability is further guaranteed by:

- Routine control of CT-models 3D.
- Regular parameters mapping between BioGears and Unity.

In addition, testing and verification activities were done to test the stability of the simulation as well as to test the stability of the output of the simulation under different conditions.

4.7 Maintainability

The system is modular by design and can be easily maintained and extended. Each of the elements functions individually:

- Creation of 3D models using tools like 3D Slicer, Blender.
- BioGears physiology Simulation engine.
- Visualization layer using Unity

The division will allow making updates or additions without affecting the entire system. For example:

- BioGears enables logic modification in the simulator.
- UI updates can be managed in Unity scripts.
- Changes in the model and its optimization are done in Blender

Clarity of script structure and standardized format can also promote maintainability that can be applied similar to FBX and DICOM.

4.8 Portability

It will be portable within popular computing platforms that support Unity and VR systems. It can run on:

- Windows-based systems
- VR goggles (e.g., Meta Quest).
- GPU systems for faster processing.

Being based on open-source tools like Unity, Blender, and BioGears, the system can be implemented on various platforms with few modifications. In addition, there is the use of standard file formats that ensure compatibility and transferring data easily.

4.9 Legacy to Modernization

Conventional technologies like CT scans and MRI give static results that are hard to visualize. These approaches in this project are revised by transforming the fixed medical imaging into a dynamic and interactive digital twin.

Key improvements include:

- CT scan data in DICOM format are converted into 3D models.
- Combining physiological simulation and visualization.
- Dynamic behavior as opposed to static observation.

The system thus bridges the gap between the old imaging system and the new digital twin technology.

4.10 Reusability

The system is designed in a manner that enables parts to be reused in other areas:

- The segmentation pipeline can be re-used to segment other organs.
- Unity visualization framework can be extended to other simulations.
- Several physiological systems can be combined with BioGears.

The modular architecture ensures that the individual components are re-usable in any projects in the same domain such as heart or brain digital twins as individual components.

4.11 Application Compatibility

The system can be integrated with several software tool and technologies:

- 3D Slicer, TotalSegmentator for conversion of imaging to models
- 3D modeling tools like Blender
- BioGears Simulation engine
- Unity Game engine for VR

It also allows connecting with VR platforms and can be extended with real-time data supplied by external sources such as sensors.

4.12 Resource Utilization

The system utilizes all computational resource effectively without any performance compromises:

- Mesh optimization methods like decimation, remeshing help decrease rendering load.
- Optimized parameter mapping is done as opposed to extensive physics calculations saving a lot of time.
- Efficient GPU utilizations

The memory consumption is regulated by:

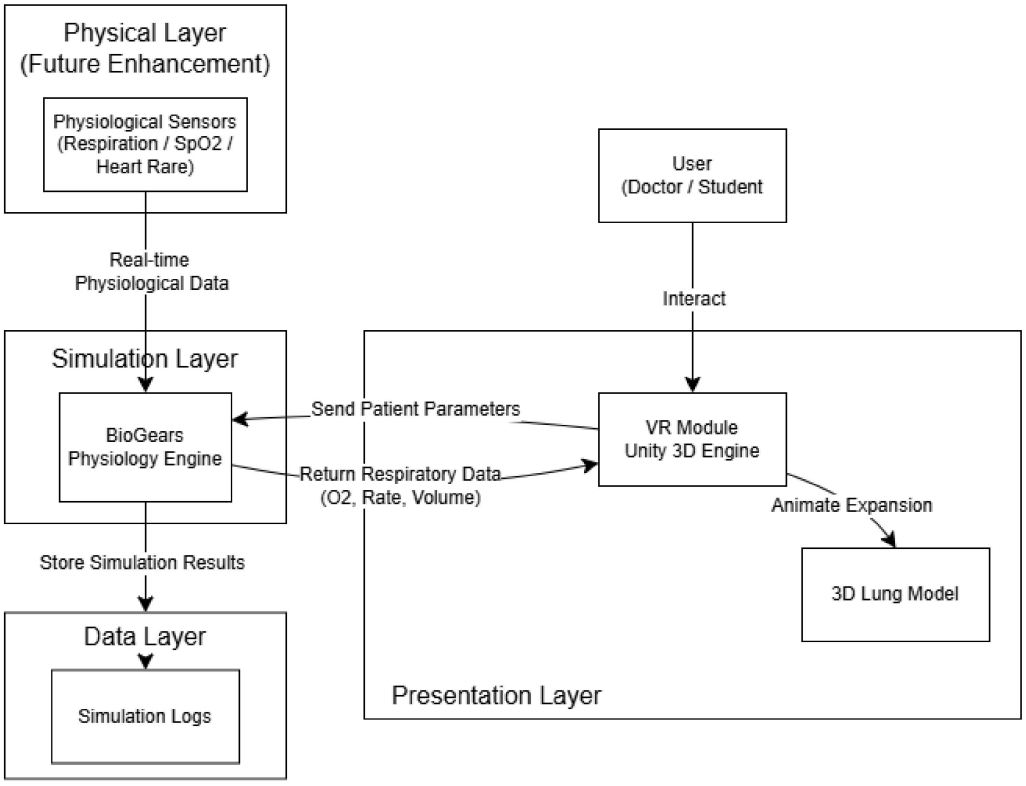
- Optimizing models 3D.
- Simulation data processed in real time without the need for having to store data in huge amounts.

CHAPTER 5

HIGH LEVEL SYSTEM DESIGN /SYSTEM ARCHITECTURE

5.1 Architecture Overview

The proposed Lung Digital Twin system is designed through a layered architecture which offers modularity, scalability and separation of concerns. The layers perform a specified role and interact with other layers through clearly defined interfaces.



The system consists of four layers:

1. Presentation Layer (Unity VR Module)

In charge of user interaction, visualization, and rendering of 3D lung model. It controls UI elements, motion and real time dynamism of physiological parameters.

2. Simulation Layer (BioGears Engine)

Functions as the calculating engine of the system. It takes the input parameters and simulates respiratory physiology and generates outputs, such as oxygen saturation, respiratory rate, and variations in lung volume.

3. Data Layer

Stores simulation data, model related information and input parameters. It helps the recording and potential analysis of outputs of simulation in the future.

4. Physical Layer (Future Enhancement)

Connects to physiological sensors like respiration monitors, pulse oximeters, enabling real-time matching of the physical and digital twin patients.

This multilayered approach improves maintainability, allowing development and scaling of components.

5.2 Logical User Groups

There are many different types of users this project is aimed towards. Some of which are mentioned below:

- Doctors and Medical Professionals
- Medical Students
- General Clinical Users

Each user group interacts with the system through the VR interface, making the system accessible and user-friendly.

5.3 Application Components

The system is making use of a number of software components:

- VR Module (Unity 3D Engine)
- BioGears Physiology Engine
- Model Processing Tools (3D Slicer / TotalSegmentator / Blender)
- Unity Scripts (C#)

All these elements collaborate together to form a complete system.

5.4 Data Components

The system uses many different forms of data in its pipeline::

- CT Scan Data (DICOM Format)
- 3D Model Files (STL, FBX)
- Physiological Parameters
- Simulation Output Data
- Log Data (Optional)

5.5 Interfacing Systems

The system makes use of various external tools and technologies:

- 3D Slicer / TotalSegmentator
- Blender
- BioGears Engine
- Unity Engine

These systems are interoperable and can exchange data with each other using standard file formats and APIs.

5.6 Data Flow

The process of systematic data flow is as follows within the system:

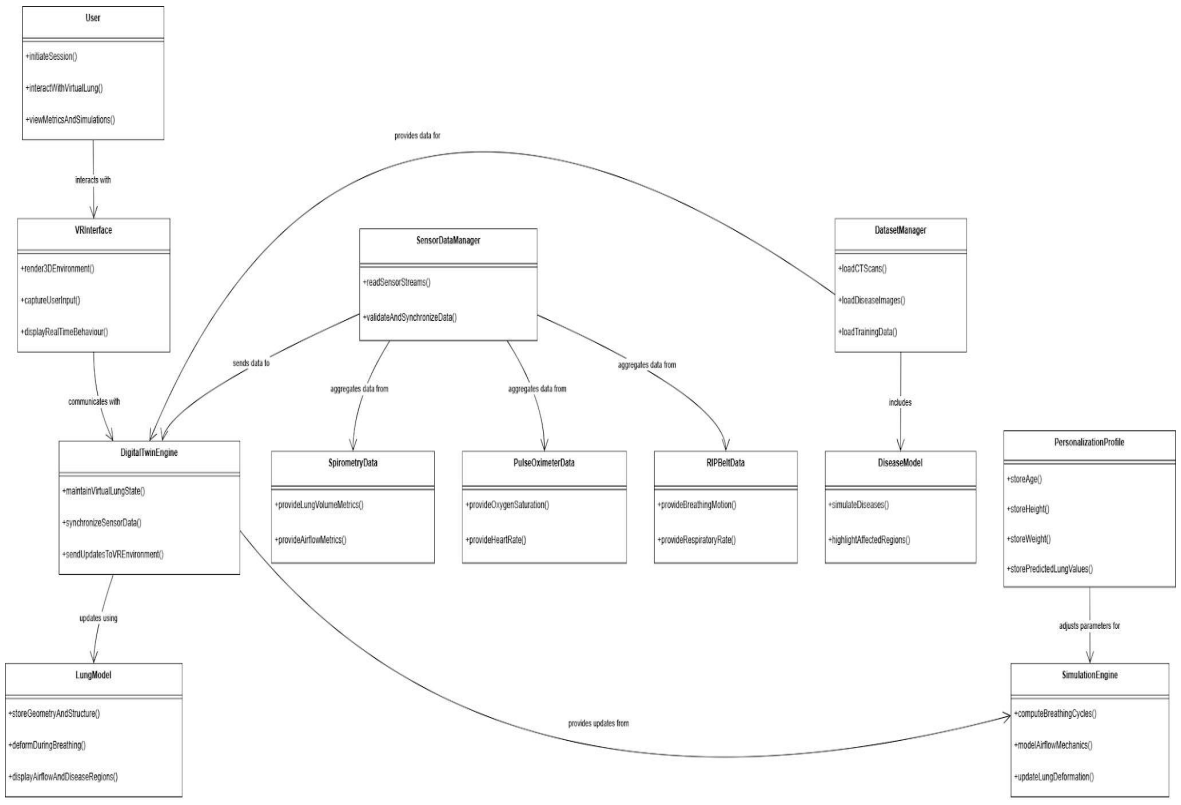
1. The user interacts with the Unity screen and selects a simulation scenario.
2. Unity takes the input parameters and passes them on to the BioGears engine.
3. The inputs are processed by BioGears which computes physiological outputs such as oxygen saturation, respiratory rate and lung volume.
4. The computed values are sent to Unity in real time.
5. Unity transforms them into the 3D lung model and re-renders the animation.
6. The user is presented with the updated visualization.
7. Simulation outcomes can be optionally stored in the data layer to be analyzed.

This constantly repeating loop allows us to interact and dynamically visualize the lungs in the VR environment.

CHAPTER 6

DESIGN DESCRIPTION

6.1 MASTER CLASS DIAGRAM



VR-Based Digital Twin Lung Simulation System - Main considerations.

1. User & VR Interface

- User works in a virtual environment.
- Can:
    - Start the simulation
    - Play with the virtual lung.
    - View in real-time.
- VR Interface:
    - Provides a 3D model of lungs.
    - Receives user inputs.
    - Shows dynamic lung behavior.

2. Digital Twin Engine (Core Component)

Central system to maintain state of the virtual lung.

- Incorporates sensor real-time information into simulation.
- Maintains VR environment.
- Maintains virtual model as a true-to-life representation.

3. Lung Model

- Represent the lung structure and functioning.
- Handles:
    - Lung geometry
    - Breathing-based deformation
    - Airflow visualization
    - Disease-affected regions

Digital Twin Engine is continually updated.

4. Sensor Data Manager

- Gathers and processes live physiological information.
- Performs:
    - Data collection
    - Validation
    - Synchronization
- Data sources include:
    - Spirometry (lungs volume and airflow)
    - Pulse oximeter (oxygen saturation & pulse rate)
    - RIP belt sensors (breathing movement & respiratory rate)
- forwards processed information to Digital Twin Engine.

6. Disease Model & Dataset Manager.

- Dataset Manager:
    - Loads medical data (CT scans, disease images, training data)
- Disease Model:
    - Simulates respiratory conditions using datasets.
    - Marks the points of affected lung.

6. Personalization Profile

- Stores user-specific parameters:
    - Age
    - Height
    - Weight
    - Lung capacity (estimated).
- Enables customized simulation for each user

7. Simulation Engine

- Performs core computations:
    - Breathing cycles
    - Airflow mechanics
    - Lung deformation
- Integrates:
    - Sensor data
    - Disease model
    - Personalization parameters
- Generates real-time simulation results.

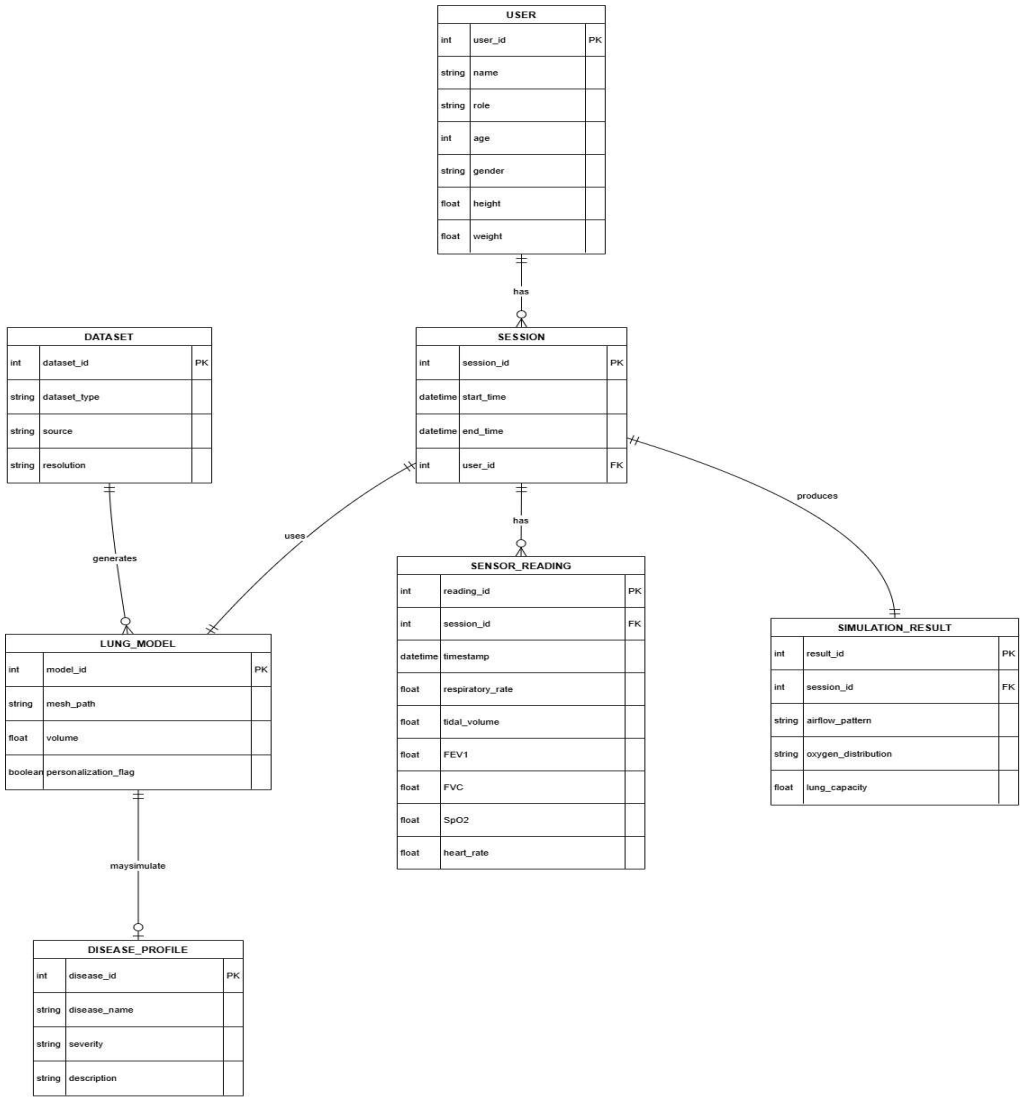
8. Overall System Workflow

- Continuous feedback loop:
    - Interaction with the user + real time data + simulation.
- Dynamically updates lung model.
- Outputs results in VR environment

9. Purpose & Applications

- Designs an interactive, customized digital twin.
- Useful for:
    - Medical visualization
    - Analysis
    - Training

6.2 ER DIAGRAM



The database design of the VR-based Digital Twin Lung Simulation System is depicted in the diagram. It describes the objects, their properties and the associations among them.

1. USER Entity

- Stores information about each user of the system.
- Attributes:
    - user_id (Primary Key)
    - name, role
    - age, gender
    - height, weight
- Multiple simulation sessions can be had by a user.

2. SESSION Entity

- Is a simulation session launched by a user.
- Attributes:
    - session_id (Primary Key)
    - start_time, end_time
    - user_id (Foreign Key)
- Relationships:
    - Every session is associated with a single user.
    - A session has several sensor readings.
    - A session produces simulation results.

3. SENSOR_READING Entity

Stores real-time physiological data obtained during a session.

- Attributes:
    - reading_id (Primary Key)
    - session_id(Foreign Key)
    - timestamp

- respiratory_rate, tidal_volume
- FEV1, FVC
- SpO2, heart_rate

- Relationship:
  - All sensor readings are associated with a particular session.

4. SIMULATION_RESULT Entity

Stores the results of an output of the simulation of a session.

- Attributes:
  - result_id (Primary Key)
  - session_id (Foreign Key)
  - airflow_pattern
  - oxygen_distribution
  - lung_capacity
- Relationship:
  - One or more simulation results are generated in each session.

5. DATASET Entity

Stores medical datasets that are used in the system.

- Attributes:
  - dataset_id (Primary Key)
  - dataset_type
  - source
  - resolution
- Relationship:

- Lung models are created using a dataset.

6. LUNG_MODEL Entity

- Reflects the 3D lung model in simulation.
- Attributes:
  - model_id (Primary Key)
  - mesh_path
  - volume
  - personalization_flag
- Relationships:
  - Produced based on datasets.
  - Application in simulation.
  - Can be related to disease patterns.

7. DISEASE_PROFILE Entity

- Stores information about different lung diseases.
- Attributes:
  - disease_id (Primary Key)
  - disease_name
  - severity
  - description
- Relationship:
  - A lung model can be used to model one or more disease profiles.

8. Relationships Summary

- One USER can have many SESSIONS.

- A SENSOR_READING can be found in many SESSIONs.

Outputs: SIMULATION_RESULTs are generated by one SESSION.

- DATASET generates LUNG_MODEL.

LUNG_MODEL can simulate DISEASE_PROFILE.

- SESSION is LUNG_MODEL-based simulation.

9. Conclusion

This ER diagram specifies a relational database to store user, session, sensor data, simulation results and medical models. It provides effective data storage and retrieval of data needed to simulate and analyze real-time lung data.

6.3 Report Layouts

The report format is a description of the way in which the simulation outcomes and physiological data could be displayed to the user in the organized and readable form within the VR environment.

The system shows real and simulated results in various visual elements. The main structure includes the 3D lung model, which is dynamically displayed as the behavior of breathing, airflows, and disease-affected areas. In addition to the visual model, other data panels are also included to show important metrics.

These measurements include respiratory rate, tidal volume, oxygen saturation (SpO2), heart rate, and lung capacity. The values are constantly updated based on sensor values on real time and simulation outputs. Graphical signatures and numerical displays are used to enhance the interpretation and understanding.

The interactive controls such as sliders allow the user to modify certain parameters such as the disease severity and exercise intensity. The alterations are immediately seen on the model of the lung and in the indicators shown.

The design will be simple and friendly to the user in a way that a user can easily track and interpret the lung behavior during the simulation process. This systematic structure of the outcomes makes it easier to use and enables to picture the respiratory conditions effectively.

6.4 External Interfaces

The system interacts with different external interfaces to enable it to acquire data, process simulation, visualize and generate the model. These interfaces allow the system to integrate real world measurements with the computational models to produce real-time and precise results.

1. Sensor Interfaces

The external medical sensors present real-time physiological information to the system. These sensors provide valuable input that is crucial in the appropriate lung simulation.

Spirometer: determines the volume of air flowing and the lung capacity.

Pulse Oximeter: Oxygen saturation (SpO2) and heart rate.

- RIP Belt(Respiratory Inductive Plethysmography): Movement of the breathing and respiratory rate is assessed.

The data available to these sensors are forwarded to Sensor Data Manager to be processed and synchronized.

2. Simulation Engine Interface

The system is integrated with the BioGears simulation engine, another model of physiological models. It is in charge of:

- Simulating breathing cycles
- Lung and airflow simulation.

- Generation of physiological outputs in reaction to input parameters.

Digital Twin Engine asks the BioGears to simulate and the output is calculated and sent back to the system where they are plotted.

3. VR Interface (Unity Engine)

The system is connected to the Unity VR platform, as the main visualization and interaction interface. It provides:

- The 3D lung model can be rendered.
- On-line changes in lung behavior.
- User interface with VR controllers.

With this interface, it is possible to engage in an immersive interaction and provide feedback in real-time.

4. Medical Data and Modeling Tools.

Lung models are made and optimized with external tools based on medical data:

- 3D Slicer: CT scan data to lung models in 3D.
- Blender: This is used to do smoothing, editing and optimization of 3D models.

These tools are used to create realistic and accurate lung structures to simulate.

5. Dataset Interface

The system involves the use of external datasets to generate models and simulate diseases. These include:

- CT scan datasets
- Disease-specific data (e.g., asthma, COPD)
- Simulation accuracy training data.

The Dataset Manager loads these datasets which are utilised to improve realism in the simulation.

6. Hardware Interface

The system is implemented on tangible hardware components, which facilitate interaction and processing:

- VR Headset to visualize the environment.

Computer system Processing, Simulation.

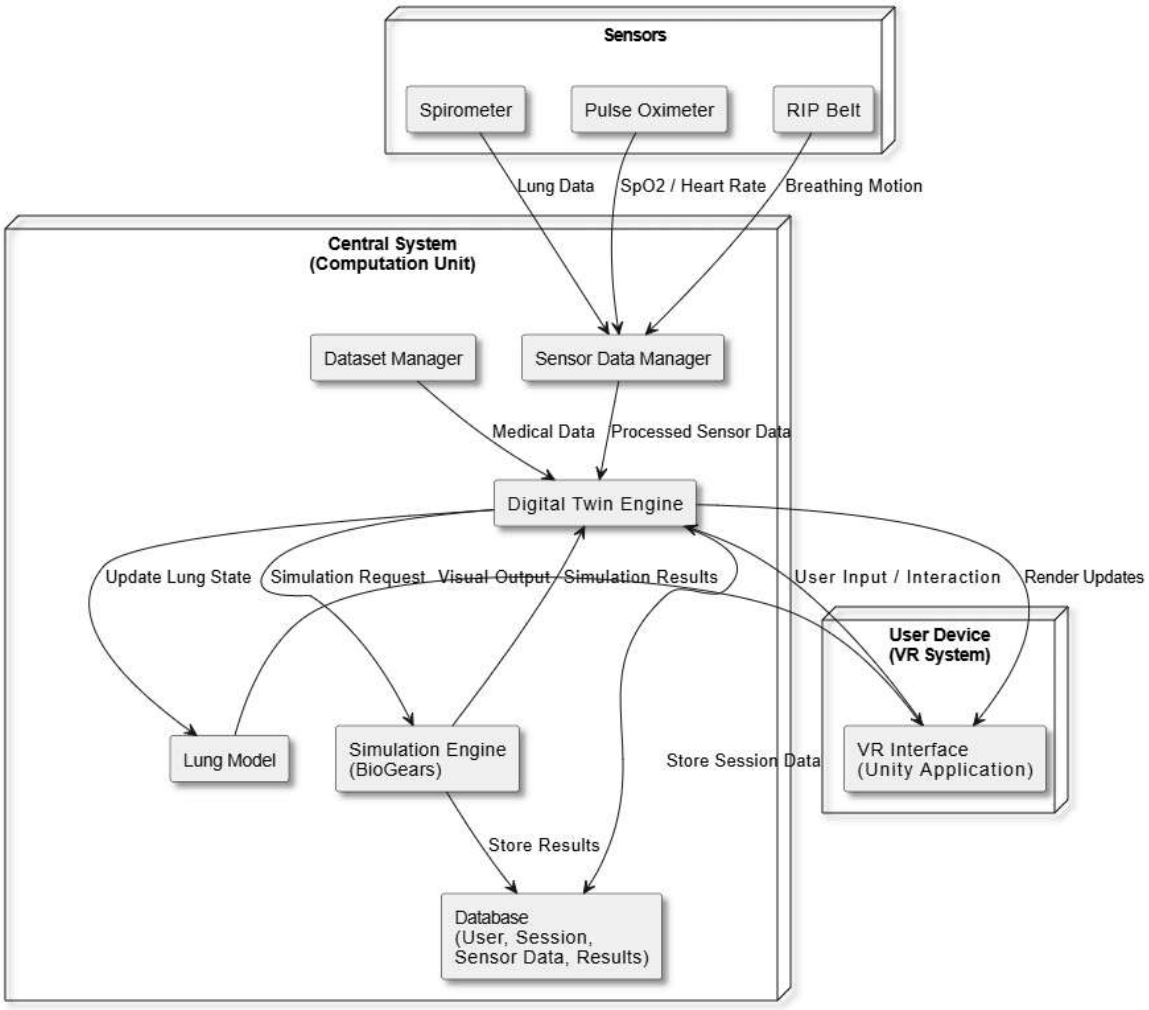
- Real-time data acquisition sensor devices.

The hardware components interact to enable the entire functionality of the system.

Conclusion

These external interfaces are also incorporated in such a way that this ensures easy interaction between hardware devices, simulation engines and visualization platforms. This will ensure that it is properly data-processed, realistically simulated, and that the digital twin lung system has an interactive user experience.

6.5 DEPLOYMENT DIAGRAM



The diagram is a combined packaging and deployment diagram of the VR-based Digital Twin Lung Simulation System. It represents physical arrangement of components in hardware devices as well as logical arrangement of software modules in the system.

1. User device (VR system)

The User Device is the VR system where the user interacts with the system. It will have the VR Interface (Unity Application), which will render the 3D lung model and receive user inputs.

The virtual environment is operated through the user by starting simulations and changing parameters. These interactions are fed into the central system, and the simulation is updated with the results fed back to the user in real time.

2. Central System (Computation Unit)

The center of all the major computations and data processing occurs on the Central System. It comprises a number of important elements:

- Digital Twin Engine:

Plays the role of the controller of the system. It controls the condition of the virtual lung, combines information across various sources, and coordinates the processes of simulation.

- Simulation Engine (BioGears):

Conducts physiological simulations including breathing cycles, airflow dynamics and lung behavior. It takes the input of the Digital Twin Engine and provide simulation outputs.

- Sensor Data Manager:

Gathers and handles real time data of outside sensors. It makes sure that the data is verified and reconciled before being utilized in the simulation.

- Dataset Manager:

Has medical information like CT scans and disease related datasets, utilized to make precise modeling and simulation.

- Lung Model:

Indicates the 3D anatomy of the lungs. It is dynamically updated according to the outputs of the simulation and takes into account changes in breathing motion and disease effects.

- Database:

Logs stores system information such as user, session, sensor and simulation outputs to be accessed and analyzed later.

3. Sensors

The Sensors node is external devices and offers real-time physiological data:

Spirometer: Airflow and lung volume.

Pulse Oximeter: This is used to measure heart rate and oxygen saturation (SpO2).

- RIP Belt: Records breathing rate and breathing motion.

These sensors continuously send data to the Sensor Data Manager in the central system.

4. Data Flow and Interaction.

The system works based on the data exchange between components:

1. The VR interface is interacted with and the input is fed to the Digital Twin Engine.

2. The sensor devices give real-time physiological data, which are processed by the Sensor Data Manager.

3. The Digital Twin Engine takes sensor and dataset inputs, and requests simulation to the Simulation Engine.

4. The Simulation Engine takes the data and gives the results back to the Digital Twin Engine.

5. The Lung Model is updated according to these results and shows change in the lung behavior.

6. New images are re-sent to the VR interface to be rendered in real time.

7. The database stores relevant data including session details and outputs of the simulation.

5. Conclusion

This diagram clearly shows the interaction of the system between user, real-time sensor data, simulation processing and visualization. It emphasizes the physical implementation of components as well as the logical implementation of software modules, allowing the system architecture to be comprehensively understood.

CHAPTER 7

TECHNOLOGIES USED

Multiple technologies were used during the implementation phase of this project. As we are still in the midst of implementation this chapter will cover the technologies we have used so far and potential technologies that we will be using. Phase 1 focused on research, literature survey, understanding datasets, and conceptual modelling. This phase, however is implementation oriented and involves many technologies like 3D modelling, rendering in real - time and scripting.

At the very heart of this project we use the Unity game engine which is the platform for building and testing the simulation. Unity was chosen as it is the most comprehensive software for modelling and simulating different objects. It has powerful real time rendering capabilities and its flexibility in handling interactive environments is what makes Unity the best tool to use for this project. This engine provides a framework where models, and materials can be mixed in a cohesive way to create and simulate dynamic behaviour. For example, our lungs breathing simulation. The lung had to continuously update the way breathing was simulated based on certain user defined parameters.

Blender also played a significant role in our project and we used it before we imported our project onto Unit. As we got our data from segmentation, we had to make a lot of changes

to our initial model to ensure the lung was realistic. This was because we had to segment manually. Later we found a way to optimize the process using an ML algorithm. We had to perform operations like remeshing, correction and flattening to improve realism. These modifiers were applied cautiously to ensure polygon count did not change and we were able to retain biological features. The model was exported as an FBX file

The C# script in Unity is the backbone of the model. The scrips links the movement and breathing of the lung and corresponds it to the parameters that we collected. We created a visual mechanism for feedback and the simulation for breathing was implemented witht he help of Biogears. Our model had to stay true to biology and replicate normal expansion and contraction while maintaining other parameters like spo2, vo2 and tidal volume

The Biogears physiology engine is a big part of our project because it helps us match the values we got with the simulation. It is a human physiology simulation system that anyone can use. It has a mathematical backbone, which is different from traditional systems.

Using data-driven research or the digital twin concept, adding Biogears greatly expands the scope of this project. Biogears is the backend that makes realistic physiological parameters, while the Unity environment handles visualization and interaction. These are things like oxygen saturation, respiratory rate, lung modeling, and how air flows through the lungs. This system can go beyond simple by adding these outputs to the Unity simulation.

Biogaears is perfect as it helps us to simulate healthy and disease lungs. We can model, lung distress and reduction in oxygen levels or disruptive functioning like COPD. This helps because we also intend to extend the scope of our project to include disease simulation. This capability aligns directly with the goals of the project to visualize how different conditions affect lung behavior. Instead of manually coding each disease

scenario, Biogears can generate realistic responses based on underlying physiological equations.

In the next phase we want to integrate biogears with unity allowing real time exchange. the lung model will respond dynamically to patient conditions. It will hence be a closer DIgitial twin

Another advantage of BioGears is its extensibility and research backing. It is widely used in academic and medical research adding credibility to this project. By leveraging BioGears the project not only improves its technical depth but also aligns itself with industry standard simulation tools. It bridges the gap between our simulation and reality.

Unity shading system also improves our visualisation. We have incorporated materials into the mesh that simulates clours making it realistic. Different colour gradients help for visual understanding and interactive and informative

The FBX file format was the best way to move assets from Blender to Unity. This format retains important information and the correct export settings had to be checked. The right setup was found after many tries. Version control was achieved using Git for project development. This is necessary for ensuring project data is safe and retraceable.

In short, the technologies used in Phase 2 made it possible to move from a static model to an interactive simulation system. Blender was used to prepare the models, Unity was used to create the environment and render it, and C# was used for logic and interaction. Together, these tools made a complete pipeline that supports the development of a digital twin framework.

CHAPTER 8

IMPLEMENTATION AND PSEUDOCODE

This Phase focused mainly on implementation and realization of the concepts that we formulated and discussed back in phase 1. Primary goal of phase 2 is to ensure these concepts from phase 1 - static lung model is converted to a real time simulation, breathing and responding to physiological inputs lung model. The first step is obtaining all the dicom files for a single CT scan and then putting it into a 3D slicer. After which we moved this file into Blender to clean the mesh obtained and smoothen the surface so its ready to be transported to unity. We exported the model in FBX format and we placed it in the scene window. We then spent significant time trying to scale and patch up the mismatches between unity and blender. Most of the time was spent coordinating between the two softwares and we hit quite a few bumps. The model was then positioned relative to the camera and lighting setup to ensure proper visibility during simulation.

Once the model was properly integrated and we were happy with it, we had to enhance its visual appearance creating a custom material for the lung mesh and adding colour and shading. This was done using reference to physiological parameters to ensure it is as realistic as possible, Creating dynamic updates.

The main part of the system is implemented using a C# script that controls and affects how th model moves and breathes. It is attached as a component and influences the lung based on changing parameters

Variables such as respiration rate and tidal volume are used. This shows that we can modify these values at run time based on our preference. This means, increasing respiration rate increases the speed of breathing cycle and change in tidal volume impacs the expansion of the lung which can be seen.

code:

```
public interface IBioGearsRespiratorySystem
{
    /// <summary>Respiration rate in breaths per minute</summary>
    float RespirationRate { get; }

    /// <summary>Tidal volume in mL</summary>
    float TidalVolume { get; }

    /// <summary>Total lung volume in mL</summary>
    float TotalLungVolume { get; }

    /// <summary>Functional residual capacity in mL</summary>
    float FunctionalResidualCapacity { get; }

    /// <summary>Inspiratory/expiratory ratio</summary>
    float InspiratoryExpiratoryRatio { get; }
```

```
    /// <summary>Current phase of breathing: 0 = start of inhale, 0.5 = start of exhale, 1 = end of cycle</summary>
    float BreathingPhase { get; }

    /// <summary>Current lung volume expansion factor (0 = fully exhaled, 1 = fully inhaled)</summary>
    float ExpansionFactor { get; }

    /// <summary>Oxygen saturation (SpO2) as fraction 0-1</summary>
    float OxygenSaturation { get; }

    /// <summary>Heart rate in BPM</summary>
    float HeartRate { get; }

    /// <summary>Whether the engine is initialized and running</summary>
    bool IsRunning { get; }

    /// <summary>Initialize the physiology engine</summary>
    void Initialize();

    /// <summary>Advance the simulation by deltaTime seconds</summary>
    void AdvanceTime(float deltaTime);

    /// <summary>Apply a condition (e.g., asthma, COPD, respiratory distress)</summary>
    void ApplyCondition(string conditionName, float severity);
}
```

2.

```
using UnityEngine;

[RequireComponent(typeof(MeshFilter))]
public class LungMeshDeformer : MonoBehaviour
{
    Mesh mesh;
    Vector3[] originalVertices;
    Vector3[] displacedVertices;

    [Header("BioGears Integration")]
      [Tooltip("Reference to the BioGears respiratory system. If null, will search on this GameObject and parents.")]
    public SimulatedBioGearsRespiratory bioGearsSystem;

    [Header("Deformation Settings")]
    [Tooltip("Maximum expansion distance in mesh units at full tidal volume")]
    public float maxExpansion = 5f;

    [Tooltip("Separate expansion multipliers for each axis (allows asymmetric breathing)")]
    public Vector3 expansionScale = new Vector3(1f, 0.8f, 1.2f);

    Vector3[] directions;

    void Start()
    {
```

```
        MeshFilter mf = GetComponent<MeshFilter>();

        if (mf == null || mf.sharedMesh == null)
        {
            Debug.LogError("LungMeshDeformer: No mesh found!");
            return;
        }

        // Find BioGears system if not assigned
        if (bioGearsSystem == null)
        {
            bioGearsSystem = GetComponent<SimulatedBioGearsRespiratory>();
            if (bioGearsSystem == null)
                bioGearsSystem = GetComponentInParent<SimulatedBioGearsRespiratory>();
            if (bioGearsSystem == null)
            {
                Debug.LogError("LungMeshDeformer: No BioGears respiratory system found!
Add SimulatedBioGearsRespiratory component.");
                return;
            }
        }

        mesh = Instantiate(mf.sharedMesh);
        mf.mesh = mesh;
        mesh.MarkDynamic();

        originalVertices = mesh.vertices;
        displacedVertices = new Vector3[originalVertices.Length];
        directions = new Vector3[originalVertices.Length];
```

```
        Vector3 center = mesh.bounds.center;

        for (int i = 0; i < originalVertices.Length; i++)
            directions[i] = (originalVertices[i] - center).normalized;

                Debug.Log($"LungMeshDeformer: Initialized with {originalVertices.Length}
vertices, connected to BioGears");
    }

    void Update()
    {
        if (mesh == null || originalVertices == null || bioGearsSystem == null) return;
        if (!bioGearsSystem.IsRunning) return;

        // Get expansion factor from BioGears (0 = exhaled, 1 = fully inhaled)
float expansion = bioGearsSystem.ExpansionFactor * maxExpansion;

        for (int i = 0; i < originalVertices.Length; i++)
        {
            Vector3 dir = directions[i];
            // Apply per-axis scaling for realistic asymmetric expansion
            Vector3 scaledDir = new Vector3(
                dir.x * expansionScale.x,
                dir.y * expansionScale.y,
                dir.z * expansionScale.z
            );
            displacedVertices[i] = originalVertices[i] + scaledDir * expansion;
        }
```

```
        mesh.vertices = displacedVertices;
        mesh.RecalculateBounds();
    }
}
```

3.

```
using UnityEngine;

/// <summary>
/// Simulated BioGears respiratory physiology model.
/// Produces realistic respiratory parameters based on published physiological data.
/// Can be replaced with real BioGears engine when DLLs are available.
///
/// References:
/// - Normal adult tidal volume: 500 mL
/// - Normal respiratory rate: 12-20 breaths/min
/// - Total lung capacity: ~6000 mL
/// - Functional residual capacity: ~2400 mL
/// - I:E ratio: 1:2 (normal)
/// - SpO2: 95-100%
/// - Heart rate: 60-100 bpm
/// </summary>
public class SimulatedBioGearsRespiratory : MonoBehaviour, IBioGearsRespiratorySystem
{
    [Header("Patient Parameters")]
    [SerializeField] private float patientAge = 30f;
```

```
    [SerializeField] private float patientWeightKg = 70f;

    [Header("Baseline Physiology")]
    [SerializeField] private float baseRespirationRate = 14f;   // breaths/min
    [SerializeField] private float baseTidalVolume = 500f;      // mL
    [SerializeField] private float totalLungCapacity = 6000f;   // mL
    [SerializeField] private float baseFRC = 2400f;             // mL
    [SerializeField] private float baseIERatio = 0.5f;          // I:E = 1:2
    [SerializeField] private float baseSpO2 = 0.98f;
    [SerializeField] private float baseHeartRate = 72f;         // bpm

    [Header("Conditions")]
    [SerializeField] [Range(0, 1)] private float asthmaSeverity = 0f;
    [SerializeField] [Range(0, 1)] private float copdSeverity = 0f;
    [SerializeField] [Range(0, 1)] private float respiratoryDistressSeverity = 0f;
    [SerializeField] [Range(0, 1)] private float exerciseIntensity = 0f;

    [Header("Variability")]
    [SerializeField] private bool enableNaturalVariability = true;
    [SerializeField] private float variabilityAmount = 0.05f;

    // Internal state
    private float simulationTime;
    private float currentPhase;
    private float currentExpansion;
    private float breathCycleTime;
    private float currentTidalVolume;
    private float currentRespirationRate;
    private float currentSpO2;
```

```
    private float currentHeartRate;
    private float currentFRC;
    private float currentIERatio;
    private float currentTotalLungVolume;
    private bool initialized;

    // Noise for natural variability
    private float breathVariation;
    private float nextBreathVariation;
    private int breathCount;

    // IBioGearsRespiratorySystem implementation
    public float RespirationRate => currentRespirationRate;
    public float TidalVolume => currentTidalVolume;
    public float TotalLungVolume => currentTotalLungVolume;
    public float FunctionalResidualCapacity => currentFRC;
    public float InspiratoryExpiratoryRatio => currentIERatio;
    public float BreathingPhase => currentPhase;
    public float ExpansionFactor => currentExpansion;
    public float OxygenSaturation => currentSpO2;
    public float HeartRate => currentHeartRate;
    public bool IsRunning => initialized;

    void Start()
    {
        Initialize();
    }

    public void Initialize()
```

```
{
    simulationTime = 0f;
    breathCount = 0;
    breathVariation = 0f;
    nextBreathVariation = Random.Range(-variabilityAmount, variabilityAmount);
    UpdatePhysiologyParameters();
    initialized = true;
    Debug.Log("[BioGears Simulated] Respiratory system initialized");
}

void Update()
{
    if (!initialized) return;
    AdvanceTime(Time.deltaTime);
}

public void AdvanceTime(float deltaTime)
{
    simulationTime += deltaTime;
    UpdatePhysiologyParameters();
    UpdateBreathingCycle(deltaTime);
}

public void ApplyCondition(string conditionName, float severity)
{
    severity = Mathf.Clamp01(severity);
    switch (conditionName.ToLower())
    {
        case "asthma":
```

```
            asthmaSeverity = severity;
            break;
        case "copd":
            copdSeverity = severity;
            break;
        case "respiratory_distress":
        case "ards":
            respiratoryDistressSeverity = severity;
            break;
        case "exercise":
            exerciseIntensity = severity;
            break;
        default:
                        Debug.LogWarning($"[BioGears Simulated] Unknown condition: {conditionName}");
            break;
    }
            Debug.Log($"[BioGears Simulated] Applied condition: {conditionName} severity={severity:F2}");
  }

  private void UpdatePhysiologyParameters()
  {
    // Respiration rate affected by conditions
    currentRespirationRate = baseRespirationRate;
    currentRespirationRate += asthmaSeverity * 10f;         // Asthma: increased rate
    currentRespirationRate += copdSeverity * 6f;            // COPD: moderately increased
      currentRespirationRate += respiratoryDistressSeverity * 16f; // ARDS: significantly increased
```

```
        currentRespirationRate += exerciseIntensity * 20f;          // Exercise: up to 34 breaths/min

    // Tidal volume
    currentTidalVolume = baseTidalVolume;
        currentTidalVolume *= (1f - asthmaSeverity * 0.3f);       // Asthma: reduced tidal volume
    currentTidalVolume *= (1f - copdSeverity * 0.25f);       // COPD: reduced
        currentTidalVolume *= (1f - respiratoryDistressSeverity * 0.4f); // ARDS: greatly reduced
    currentTidalVolume += exerciseIntensity * 1500f;         // Exercise: up to ~2000 mL

    // Functional residual capacity
    currentFRC = baseFRC;
    currentFRC *= (1f + copdSeverity * 0.4f);              // COPD: hyperinflation
        currentFRC *= (1f - respiratoryDistressSeverity * 0.2f); // ARDS: decreased compliance

    // I:E ratio
    currentIERatio = baseIERatio;
    currentIERatio *= (1f - asthmaSeverity * 0.3f);          // Asthma: prolonged expiration
    currentIERatio *= (1f - copdSeverity * 0.4f);           // COPD: prolonged expiration

    // SpO2
    currentSpO2 = baseSpO2;
    currentSpO2 -= asthmaSeverity * 0.08f;
    currentSpO2 -= copdSeverity * 0.06f;
    currentSpO2 -= respiratoryDistressSeverity * 0.15f;
    currentSpO2 = Mathf.Clamp(currentSpO2, 0.7f, 1f);
```

```
        // Heart rate
        currentHeartRate = baseHeartRate;
        currentHeartRate += asthmaSeverity * 20f;
        currentHeartRate += respiratoryDistressSeverity * 30f;
        currentHeartRate += exerciseIntensity * 80f;            // Exercise: up to 150+ bpm

        // Total lung volume (FRC + current tidal volume contribution)
        currentTotalLungVolume = currentFRC + currentTidalVolume * currentExpansion;

        // Apply natural variability
        if (enableNaturalVariability)
        {
            float variation = Mathf.Lerp(breathVariation, nextBreathVariation, currentPhase);
            currentRespirationRate *= (1f + variation * 0.5f);
            currentTidalVolume *= (1f + variation);
        }

        breathCycleTime = 60f / currentRespira...

<...etc...>
```

CHAPTER 9

CONCLUSION OF CAPSTONE PROJECT PHASE - 2

Phase 2 is the most important stage in the development of our VR based 3D Lung Digital Twin Visualisation System. It bridges the gap between our phase 1 which was conceptual modelling and the phase 3 part of the project which involves complete implementation.

The significant achievement of this phase was the start of implementation. This niche domain posed obstacles that we had not anticipated and navigating through this was an unexpected win. We successfully obtained a working dataset and were able to extract the lungs from a complete chest CT. After which we put it into Bender and on significant mesh cleaning it was ready to be imported into Unity. On matching the inconsistencies we faced between the two software, we were able to obtain our final lung model. Lastly, we transferred this model onto the Meta Quest while the biological parameters could be modified and changed on Unity.

One of the most significant achievements of this phase was the successful integration of the lung model into a real time interactive environment. The implementation of breathing animation was a big stepping stone as it transformed the model from a static system into a dynamic system. By simulating inhalation and exhalation using mathematical functions, the system was able to simulate the natural way of breathing in humans in a visually convincing way. Helps in education.

An important accomplishment that we achieved was parameter driven simulation. Including parameters like Spo2, Vo2 we can see how our model responds to these

changing parameters. This makes our project a digital twin. This pimples our lung performs and behaves as a true digital twin

Phase 2 has been a big success now that we have demonstrated feasibility or creating an interactive system. We have started the technical implementation which will be useful for phase 3. We are confident this project will evolve into a comprehensive digital twin in the next phase

CHAPTER 10
PLAN OF WORK FOR CAPSTONE PROJECT
PHASE-3

The next phase we have to carry forward what we have done in the previous phases. We wish to improve realism, interaction and transition a basic simulation into a true digital twin

Improving physiology is a key goal and the current implementation relies on mathematical models to simulate breathing and the next phase we will replace that with more accurate representations of lung mechanics. We aim to include parameters like airflow resistance and gas exchange process. Another major development is disease simulation. We aim to extend the system to model conditions like COPD, pulmonary diseases, asthma etc. This means we have to modify lung behaviour to reflect restricted airflow, reduced plasticity and reduction in oxygen efficiency.

We intend to incorporate a very interactive environment including sliders, input fields and dashboards to allow users to interact with the VR and observe the influence of changing parameters on the lung.

We aim to complete the final project by the end of phase. Which means, we will incorporate sliders and UI in the Meta Quest so individuals can visualise the changing effects on the lung. We aim to optimize the pipeline to reduce the time it takes from the input to the final output. We aim to incorporate disease simulation and create a much more interactive environment which is beneficial not only for doctor patient communication but also for educational purposes.

Finally, validation and testing will be conducted to assess the accuracy and reliability of the system. This will involve comparing our outputs with true medical data.

Phase 3 transitions our project from a static system to a dynamic interactive Digital twin, We are going to focus on accuracy and integration. Focusing mainly on real world applications